

Testing Your Databases with DbUnit

DbUnit is ideal as an add-on testing framework when developing applications in Java-based languages, which rely heavily on databases. This article assumes that the reader is aware of basic testing concepts, and tries to present DbUnit as a useful addition to the testing process.

Test-driven development is quite important these days, because of the fact that we need to put our requirements ahead of what we presume, and produce code that 'just works' for the given specification. So, if it's a database-backed application in Java, or any JVM-based language that you're working on, you should probably try out DbUnit for database testing, and that is precisely what we're going to do in this article.

Sure, performing a basic SELECT query and some mock objects might just work for some of you, but testing against a live database is as critical as the application itself. It is not only important in ironing out those bugs that crop up in the long run, but also gives you a feel of how the actual data affects your application in the real world. Ideally, DbUnit should come into the picture when you are performing integration testing, although you are absolutely free to use it in whatever manner you like.

When you are writing tests, you always want them to be repeatable, so that their results are not affected by the time at which they are run. But unfortunately, for applications that rely on data, as opposed to code, the data in their database does change quite often, over time. This is why you might want each of your test cases to handle their own data, and probably even carry a test dataset along with them. Also, another factor to be taken care of while writing tests is their inter-dependence; you surely do not want the results of one test affecting the results of the others—you shouldn't have to order your tests in a particular manner to prevent them somehow going wrong. These are some of the reasons for using DbUnit.

DbUnit features

DbUnit performs the following basic functions (represented in Figure 1) to help you test with databases:

- It exports data from the database into flat XML files, so that data for each test can be loaded from these files.

- It cleans or deletes any corrupted/changed data (in the database) before performing the test.
- It loads fresh data pertaining to the current test into the database.
- It makes comparisons between the actual data in the database, and the expected datasets, so as to determine the success or failure of a particular test.
- It deletes any changes made after the test is complete—although the developers discourage this, so that any corruption or unwanted changes can be tracked down, instead of being overwritten by DbUnit.

Setting up the environment

DbUnit was developed as an extension to be used along with JUnit; I have also seen it being used in many different ways, such as with TestNG, and sometimes with Selenium. We will look at a basic example and run a simple test case with the help of JUnit, in the Eclipse IDE, along with a MySQL database. At the very least, we will need the following libraries in our CLASSPATH before trying to write any tests, so open up an Eclipse project, and add the following *jar* archives to your CLASSPATH:

- JUnit 4
- DbUnit 2.4.8
- mysql-connector-java-5.1.13
- slf4j-api-1.6.1
- slf4j-simple-1.6.1

Since Eclipse already includes JUnit 4, it will automatically add it while creating a new test case, and you only need to get hold of the last four on the list. The last two dependencies are necessary because DbUnit uses *slf4j* (a simple logging facade for Java) for logging purposes. You may use any driver (appropriate to your particular database system) instead of the MySQL driver that we have used, but make sure that your database is supported. At the moment,